St. John College of Engineering and Management
Autonomous Institute
(A Christian Religious Minority Institution)
Approved by AICTE and DTE, Affiliated to University of Mumbai / MSBTE
DTE Code : 3218 AICTE Permanent ID : 1-4790201
NAAC Accredited with Grade 'A+', Three Programs NBA Accredited

CAPSTONE PROJECT REPORT

SmartPharm

Review Assisted Pharmacy Management with Computer Vision and Intelligent Inventory

Submitted in partial fulfilment of the requirements of the degree of

Bachelor of Engineering

in

Artificial Intelligence and Machine Learning

by

Neev Borana [07] [EU1246063]
Romit Meher [32] [EU1226023]
Rohit Gorhekar [15] [EU1246019]

Under the Guidance of

_____

Department of Artificial Intelligence and Machine Learning

ST. JOHN COLLEGE OF ENGINEERING AND MANAGEMENT
UNIVERSITY OF MUMBAI

Academic Year 2026-27

Certificate of Approval

This is to certify that **Neev Borana [EU1246063], Romit Meher [EU1226023] and Rohit Gorhekar [EU1246019]**, students of the Department of Artificial Intelligence and Machine Learning, St. John College of Engineering and Management, have submitted the capstone project entitled **SmartPharm Review Assisted Pharmacy Management with Computer Vision and Intelligent Inventory** for academic evaluation during the academic year 2026-27.

This certificate is to be completed and signed by the authorised college representatives following examination and approval.

Project Guide

Name and designation: _____

Signature: _____ Date: _____

Head of Department

Name and designation: _____

Signature: _____ Date: _____

Principal

Name and designation: _____

Signature: _____ Date: _____

External Examiner

Name and designation: _____

Signature: _____ Date: _____

Declaration

We declare that this report describes our capstone project SmartPharm. Ideas, methods and published work used from other authors are acknowledged through numbered references. Open-source libraries, cloud platforms and model services are identified separately from the application functionality developed for this project.

We understand that academic integrity requires accurate reporting of implementation, observations and test outcomes. Demonstration transactions are fictional records used to explain system behaviour. They are not evidence of actual pharmacy sales, clinical validation or commercial deployment at a licensed pharmacy.

We further declare that this submission does not present assistance from optical character recognition or generative AI as a substitute for professional verification. Final medicine selection and dispensing decisions remain subject to an authorised pharmacist and applicable requirements.

Student	PID	Signature
Neev Borana	EU1246063	_____
Romit Meher	EU1226023	_____
Rohit Gorhekar	EU1246019	_____

Place: _____

Date: _____

Acknowledgement

We thank St. John College of Engineering and Management and the Department of Artificial Intelligence and Machine Learning for the academic setting in which this capstone project has been developed.

We acknowledge the guidance of our project supervisor, the departmental faculty and the laboratory staff. Their review and practical feedback support the organisation, implementation and presentation of our work.

We acknowledge the researchers whose work on optical character recognition, document understanding and retrieval augmented generation informed the technical study. We also acknowledge the maintainers of the open-source frameworks and tools listed in this report.

We thank our families and classmates for their encouragement during development and demonstration preparation.

Neev Borana
Romit Meher
Rohit Gorhekar

Department of Artificial Intelligence and Machine Learning
Academic Year 2026-27

Abstract

SmartPharm is a web-based pharmacy management project that connects medicine stocking, batch-level inventory, point-of-sale billing and operational review. It addresses the difficulty of managing medicine identity, changing batch prices, expiry dates, shelf locations and sales records across separate manual workflows. The design separates a medicine catalogue entry from a manufactured batch and from the quantity physically stored at a location.

The implemented system uses a Next.js and React frontend with a Flask and SQLAlchemy backend. Aiven MySQL stores business transactions, while Aiven PostgreSQL supports AI reference and vector data. Barcode input, camera capture and packaging OCR provide assisted identification. Candidates are checked against the store catalogue and presented for human review. Prescription digitisation provides editable draft rows, and supplier invoice processing supports review before physical stock is accepted.

Billing applies first-expiring-first-out selection and records stock deductions with the sale transaction. Dashboard functions show stock and expiry alerts, purchase drafts and recent invoice activity. Demand estimation and reorder calculations support planning, with a moving-average fallback for sparse sales history. PWA support provides installation controls, bounded public-asset caching and a safe reconnect page.

The classroom demonstration uses eight medicines represented by sixteen front-and-back package photographs, with labelled synthetic activity for dashboard presentation. Verification combines automated tests, read-only checks against the deployed service, browser layout checks and direct observation of the recognition workflow. Recognition output remains review-assisted; benchmark-level identification accuracy and physical-device performance require separate evaluation.

Keywords Pharmacy management, FEFO, OCR, barcode, inventory, human verification, PWA, document digitisation.

Contents

Section	Page
Certificate of Approval	2
Declaration	3
Acknowledgement	4
Abstract	5
List of Figures and Abbreviations	7
Chapter 1 Introduction	8-9
Chapter 2 Literature Review	10-11
Chapter 3 Proposed Methodology and System Design	12-24
Chapter 4 Project Planning	25
Chapter 5 Resources and Feasibility	26
Chapter 6 Verification and Frontend Screens	27-37
Chapter 7 Sustainability and Impact	38
Chapter 8 Conclusion and Future Scope	39
References	40-41
Appendix Examination and Demonstration Guide	42

The diagram sequence includes activity, DFD 0, DFD 1, DFD 2, system flow, flowchart and functional flow. Eight actual frontend screens appear in Chapter 6.

List of Figures and Abbreviations

Figure	Description	Page
3.1	System architecture	13
3.2	Activity diagram with responsibility lanes	14
3.3	DFD 0 context diagram	15
3.4	DFD 1 major business processes	16
3.5	DFD 2 decomposition of POS billing	17
3.6	System flow and deployment boundaries	18
3.7	Recognition decision flowchart	19
3.8	Functional flow through stocking and billing	20
3.9	Core entity relationship model	21
6.1	Test case and verification flow	27
6.2-6.9	Eight frontend screenshots	30-37

Abbreviations

AI Artificial Intelligence; **API** Application Programming Interface; **DFD** Data Flow Diagram; **FEFO** First Expiring First Out; **GST** Goods and Services Tax; **JWT** JSON Web Token; **ML** Machine Learning; **OCR** Optical Character Recognition; **POS** Point of Sale; **PWA** Progressive Web App; **RAG** Retrieval Augmented Generation; **ROI** Region of Interest; **UML** Unified Modeling Language.

List of Tables

1.1 Functional requirements (p. 9); 2.1-2.2 Literature comparison (pp. 10-11); 3.1-3.2 API inventory (pp. 23-24); 4.1 Milestones and 4.2 Risks (p. 25); 5.1 Resources (p. 26); 6.1 Test cases (p. 28); 6.2 Demo data (p. 29); 7.1 SDG mapping (p. 38); R.1 Evidence inventory (p. 41); A.1 Demo sequence (p. 42).

CHAPTER 1

Introduction

1.1 Background

A physical pharmacy must identify the correct product, locate saleable stock, respect batch expiry, record payment and retain an auditable history. A brand may have several strengths and package sizes, while successive batches can have different purchase prices and maximum retail prices. A single product-level stock number cannot explain these differences.

Manual registers and disconnected spreadsheets require staff to repeat entry of medicine names, batch numbers and invoice quantities. This raises the chance of inconsistent records and makes expiry review difficult. Computer vision can reduce typing, but printed foil, glare, rotated text and handwriting make automatic recognition uncertain. SmartPharm therefore combines assisted input with explicit review rather than making recognition the authority for a sale.

1.2 Problem Statement

The project problem is to connect medicine entry, physical inventory and billing in a single traceable workflow while allowing staff to use barcodes, images or manual search. The system must preserve the medicine-batch-location hierarchy, stop invalid stock operations and make failed recognition recoverable without guessing a product.

1.3 Need of the Project

The project provides an integrated academic prototype for Indian pharmacy counter operations. It demonstrates how database constraints, role-based access, transaction handling and human verification can be combined with OCR and inventory estimates. Dashboard alerts provide a practical view of expiry and replenishment needs using stored transaction records.

1.4 Scope Boundary

The implemented scope covers a pharmacy chain installation with store-scoped operations. It includes management assistance and record keeping. It does not establish a clinically validated diagnosis, prescribe treatment, guarantee medicine equivalence, or certify regulatory compliance. The classroom dataset is deliberately limited to eight active demonstration medicines.

Objectives and Requirements

1.5 Project Objectives

The principal objective is to build a connected pharmacy workflow that remains usable when optional recognition services cannot produce a reliable result. Specific objectives are to maintain distinct medicine, batch and inventory records; support barcode and review-assisted photo entry; apply FEFO billing; make purchase acceptance and returns traceable; expose stock and expiry information; and verify the important stock and payment invariants.

ID	Functional requirement	Expected behaviour
FR1	Authentication and staff roles	Allow authorised actions for owner, manager, pharmacist, cashier and inventory staff.
FR2	Medicine and stock entry	Record identity, ingredients, aliases, batch expiry, prices, location and quantity separately.
FR3	Assisted recognition	Read barcode or package text; show candidate for review and allow manual correction.
FR4	Billing and returns	Choose saleable batches, commit sale and stock changes together, restrict eligible returns.
FR5	Procurement and review	Keep purchase drafts separate from accepted physical intake.
FR6	Operational insight	Show recorded sales, low stock, near expiry and explainable replenishment estimates.

Table 1.1 Functional requirements

1.6 Nonfunctional Requirements

The interface should remain readable on phone and desktop screens. Authentication and sensitive business responses must not enter a general offline page cache. Production access uses HTTPS, server-side validation and environment-managed secrets. Recognition and network failures must leave existing records unchanged until a user confirms a valid operation. These are design requirements; full load and security certification remain separate activities.

CHAPTER 2

Literature Review

The survey covers seven primary research papers on scene text recognition, OCR pipelines, document structure and grounded generation. Foundational papers are included because their architectural ideas remain relevant to packaging and invoice processing. The findings below describe the cited research, not measured accuracy of SmartPharm.

Paper and year	Method and finding	Relevance and remaining gap
[1] Shi, Bai and Yao, 2015	CRNN combines convolutional features, recurrent sequence modelling and transcription for image text.	Explains image-to-text recognition. A recognised sequence still needs medicine identity, strength and stock validation.
[2] Liao et al., 2019 / AAAI 2020	Differentiable binarisation learns an adaptive threshold within scene text detection.	Relevant to locating text regions. Detection alone cannot establish which medicine is present.
[3] Du et al., 2020	PP-OCR combines lightweight detection, direction classification and recognition components.	Supports a practical OCR pipeline. Reflective foil and distorted package photographs still require application-level handling.
[4] Li et al., 2021 preprint	TrOCR combines pretrained image and text Transformers for printed and handwritten text.	Motivates handwriting assistance. Transcription does not establish a safe dosage or resolve every ambiguous brand.

Table 2.1 Literature comparison part one

2.1 Implications for Recognition

These studies separate text detection from text recognition and demonstrate the value of learned sequence models. SmartPharm uses an available OCR toolkit rather than claiming to reproduce each research model. The application adds catalogue matching and a review step because a plausible OCR string is insufficient evidence for dispensing. Barcode lookup is preferred when a known identifier can be decoded.

Document Understanding and Research Gap

Paper and year	Method and finding	Project relevance and gap
[5] Huang et al., 2022	LayoutLMv3 uses unified text/image masking and word-patch alignment for document representations.	Shows why invoice coordinates and visual structure matter. The project uses OCR layout parsing; it does not implement LayoutLMv3.
[6] Cui et al., 2025	PaddleOCR 3.0 describes multilingual OCR, structured parsing and information extraction components.	Supports the choice of a maintained OCR ecosystem. Published benchmarks cannot be transferred to a local pharmacy dataset.
[7] Lewis et al., 2020	RAG combines a pretrained generator with retrieved external information.	Motivates grounding the copilot in retrieved records. Retrieval does not guarantee correctness or grant permission to change stock.

Table 2.2 Literature comparison part two

2.2 Research Gap

The papers concentrate on recognition or representation quality. A pharmacy workflow also needs store-specific identifiers, expiry-aware stock, transaction integrity and a clear response to ambiguous input. OCR benchmark success does not address whether a scanned medicine is currently stocked, whether its batch is expired, or whether a return should restore inventory.

2.3 Project Contribution

SmartPharm integrates existing recognition capabilities with the operational medicine-batch-inventory model. Its application contribution is the connected review flow from stocking to billing, including barcode aliases, manual recovery, purchase verification, FEFO allocation and audit records. This is an engineering integration contribution; no novel OCR architecture or clinical accuracy result is claimed.

2.4 Evaluation Direction

A later recognition study should separate training and test photographs by original package and capture session, report rejection rate as well as accepted-match precision, and test rotated, cropped, low-light and glare examples. The same photograph used as both a reference and a test is unsuitable evidence of generalisation.

CHAPTER 3

Proposed Methodology

3.1 Proposed System

A staff member signs in to the frontend. The application obtains authorised business data through Flask APIs. Business writes are validated and recorded in MySQL; AI references are maintained separately in PostgreSQL. Optional services support document reading and search, while the authoritative stock state remains in the business database.

3.2 Working Principle

Stocking: enter or read a product label, search the catalogue and optional reference sources, review the medicine identity, then confirm batch and location quantities. A reference photo or matching result does not by itself create saleable stock.

Billing: scan a known barcode, read packaging, or search manually. Review the selected medicine, enter a quantity and collect payment information. The server validates stock and batch expiry, allocates eligible inventory in FEFO order and commits the transaction. A pending local bill is not treated as a synchronised sale.

Document review: invoice OCR produces editable candidate rows before physical intake is accepted. Prescription capture produces draft medicine rows, with ambiguous fields left for review. Optional Google handwriting processing is a separate third-party transfer of the selected image.

3.3 Development Method

The implementation follows an incremental workflow: inspect existing routes and models, define the stock invariants, implement the service/API boundary, connect the UI, add focused regression tests and verify the deployed workflow. Unit tests use isolated test databases; read-only service checks inspect the Aiven-backed deployment. UI checks inspect both narrow and wide layouts.

3.4 Diagram Notation

The following figures use labelled arrows. In DFDs, rectangular entities are outside the system, rounded processes transform data, and open-ended stores represent logical persisted records. The DFD stores are logical groups within MySQL, not additional physical databases. Activity and flow diagrams describe intended control paths, including rejection and manual recovery.

System Architecture

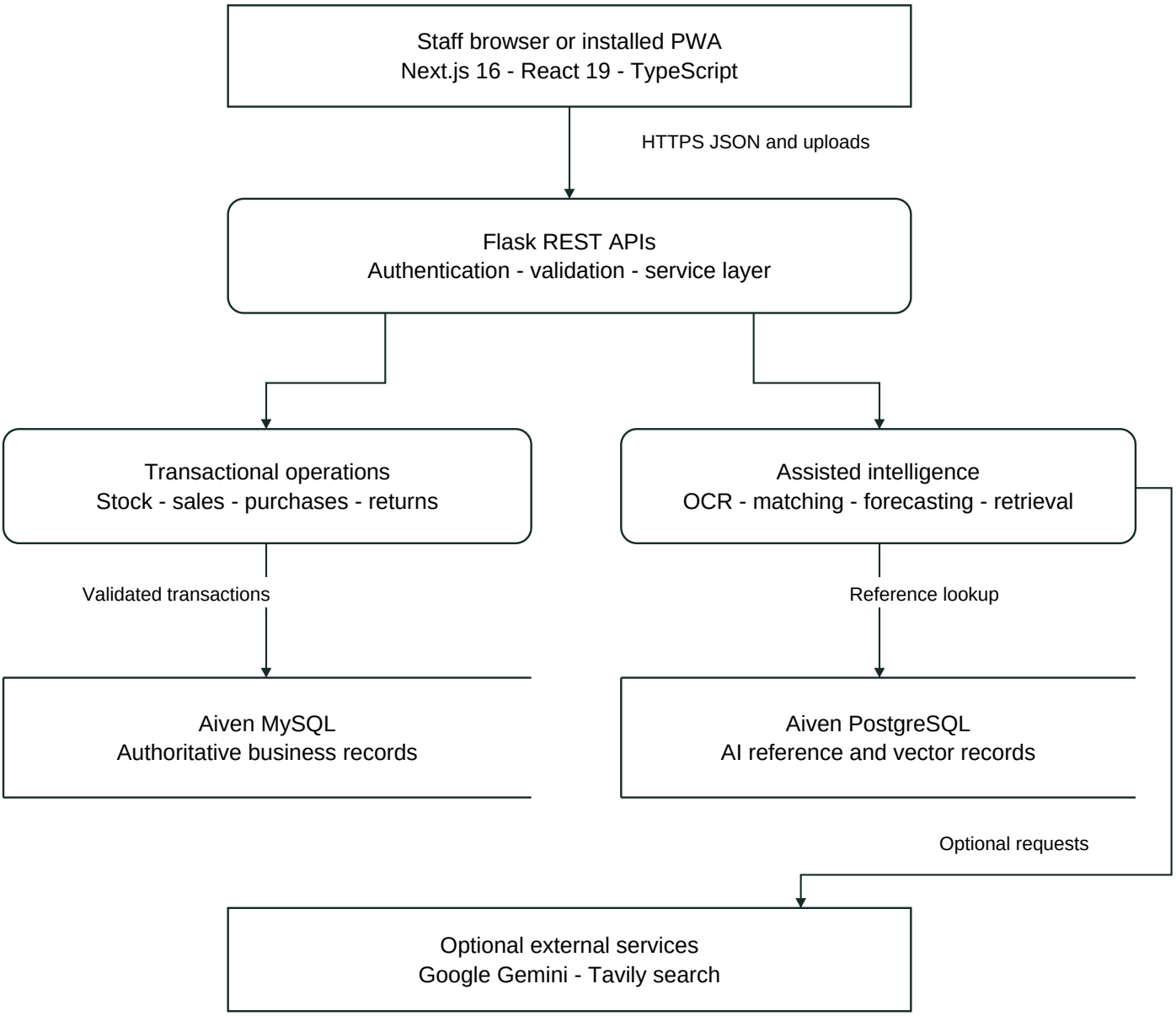


Figure 3.1 System architecture

The browser does not connect directly to either database. MySQL is the business source of truth; PostgreSQL supports reference retrieval. External AI and search responses return through server-side validation and user review.

Activity Diagram

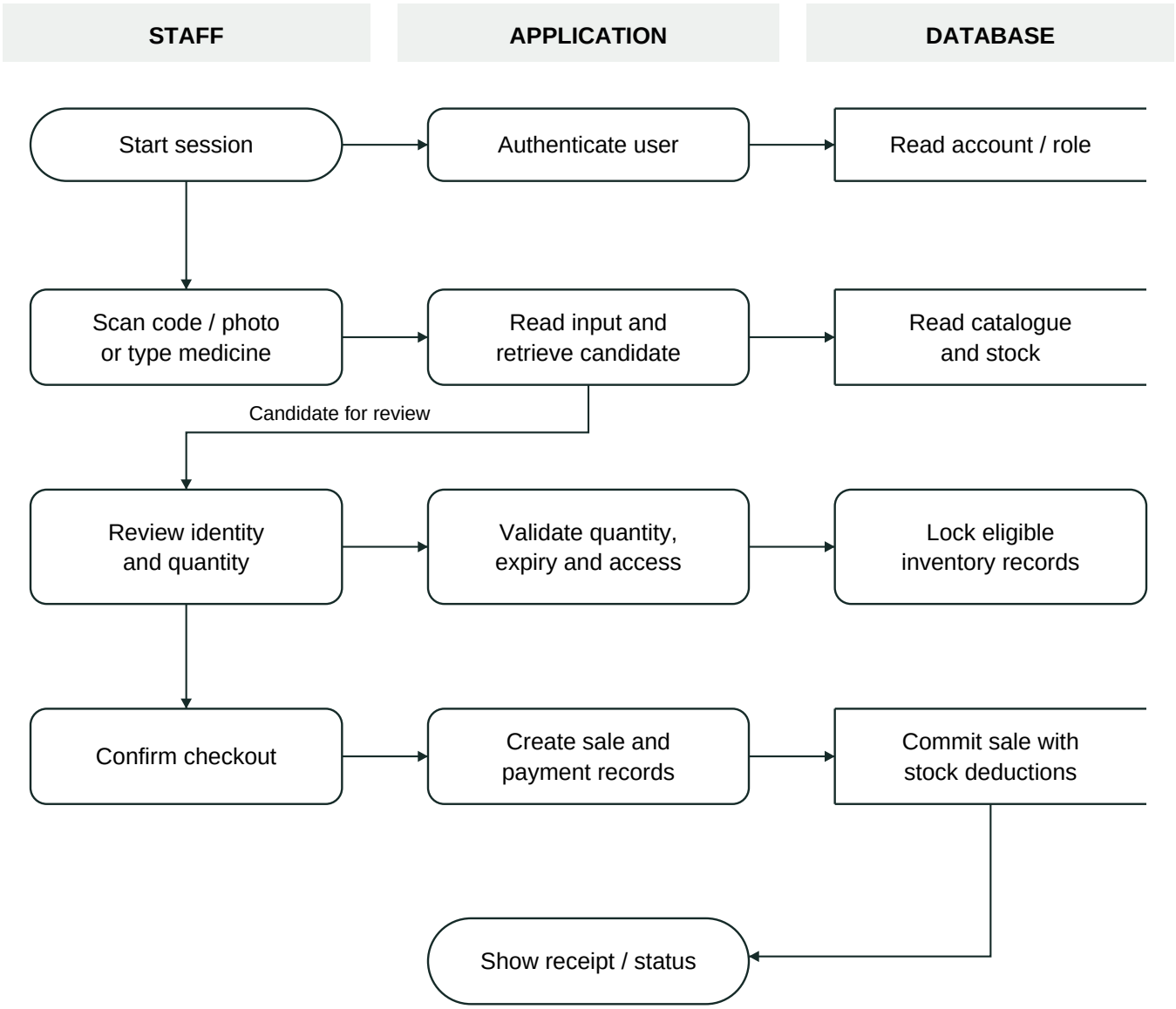


Figure 3.2 Activity diagram for a reviewed POS sale

Uncertain recognition returns to manual search or recapture before confirmation. Validation failure stops the write and displays a correction message. A receipt is shown only after the API returns the resulting transaction state.

DFD 0 Context Diagram

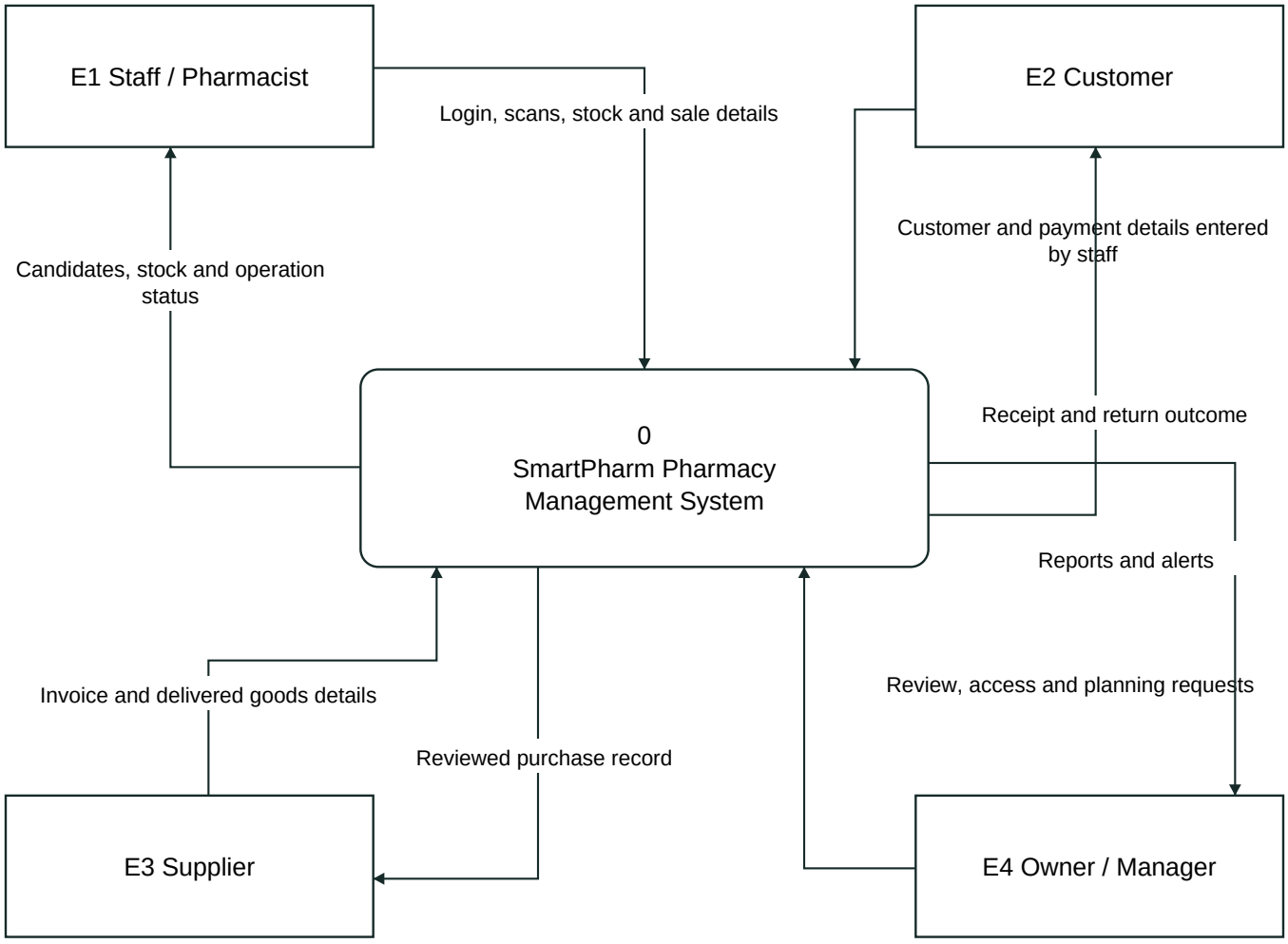


Figure 3.3 DFD 0 context diagram

The system boundary contains pharmacy record processing. Customer and supplier information is entered by authorised staff; these arrows do not imply an online customer portal, payment gateway settlement or automatic supplier ordering.

DFD 1 Major Processes

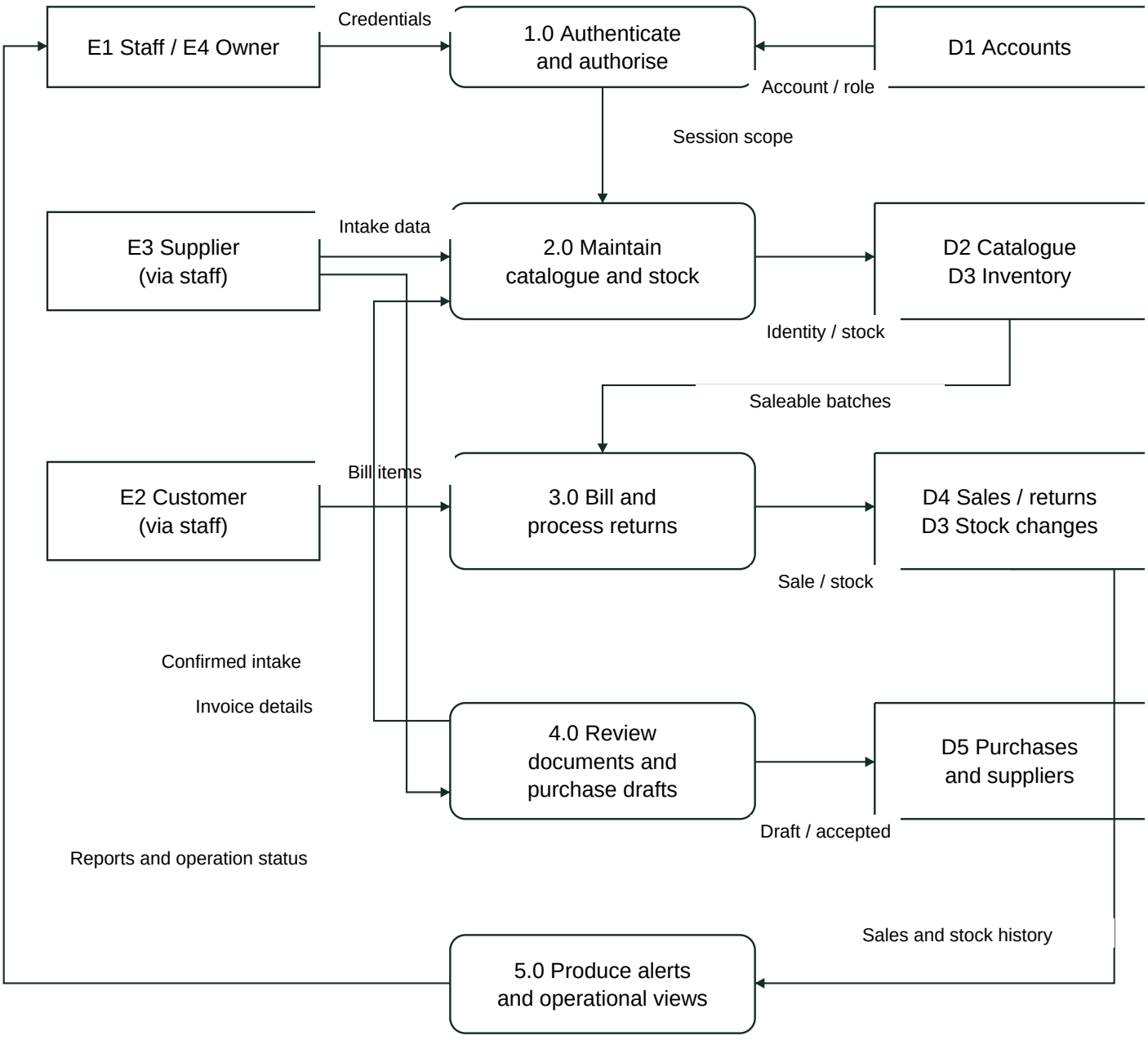


Figure 3.4 DFD 1 major processes

D2 stores medicine identities and aliases. D3 stores batches, quantities and locations. D4 stores sales, tender records and returns. Processes share store-scoped records; separate DFD stores do not indicate separate business databases.

DFD 2 POS Billing Decomposition

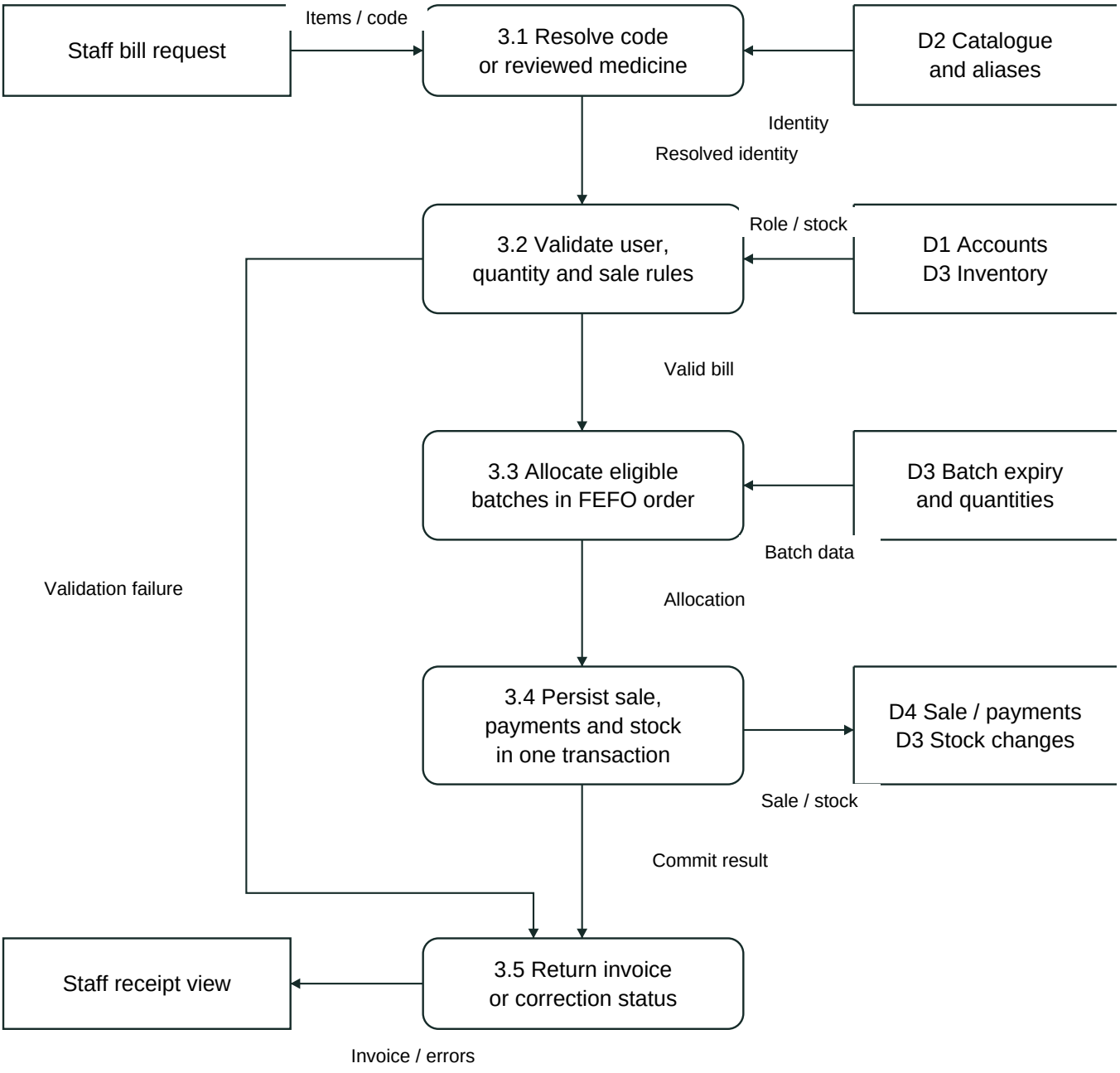


Figure 3.5 DFD 2 decomposition of process 3.0 billing

The branch to a correction status performs no sale write. The successful branch updates inventory and financial records atomically. Return processing is a separate guarded operation against eligible previously sold items.

System Flow

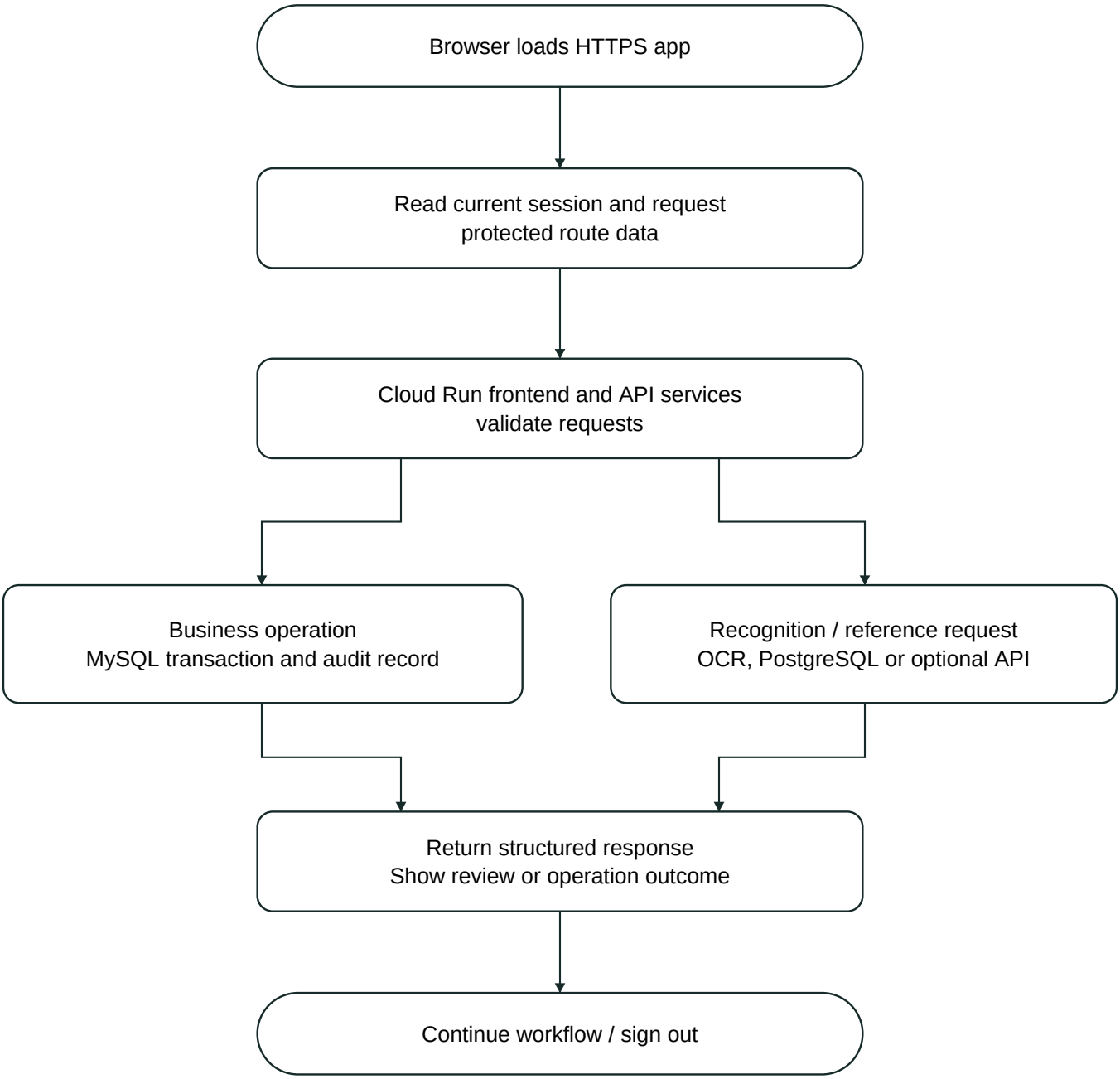


Figure 3.6 System flow across deployment boundaries

The public PWA worker caches only permitted static assets and a reconnect screen. Protected navigation is fetched freshly, and a cold offline launch shows the reconnect fallback. Existing account-scoped POS queue behaviour remains separate from service-worker caching.

Recognition Flowchart

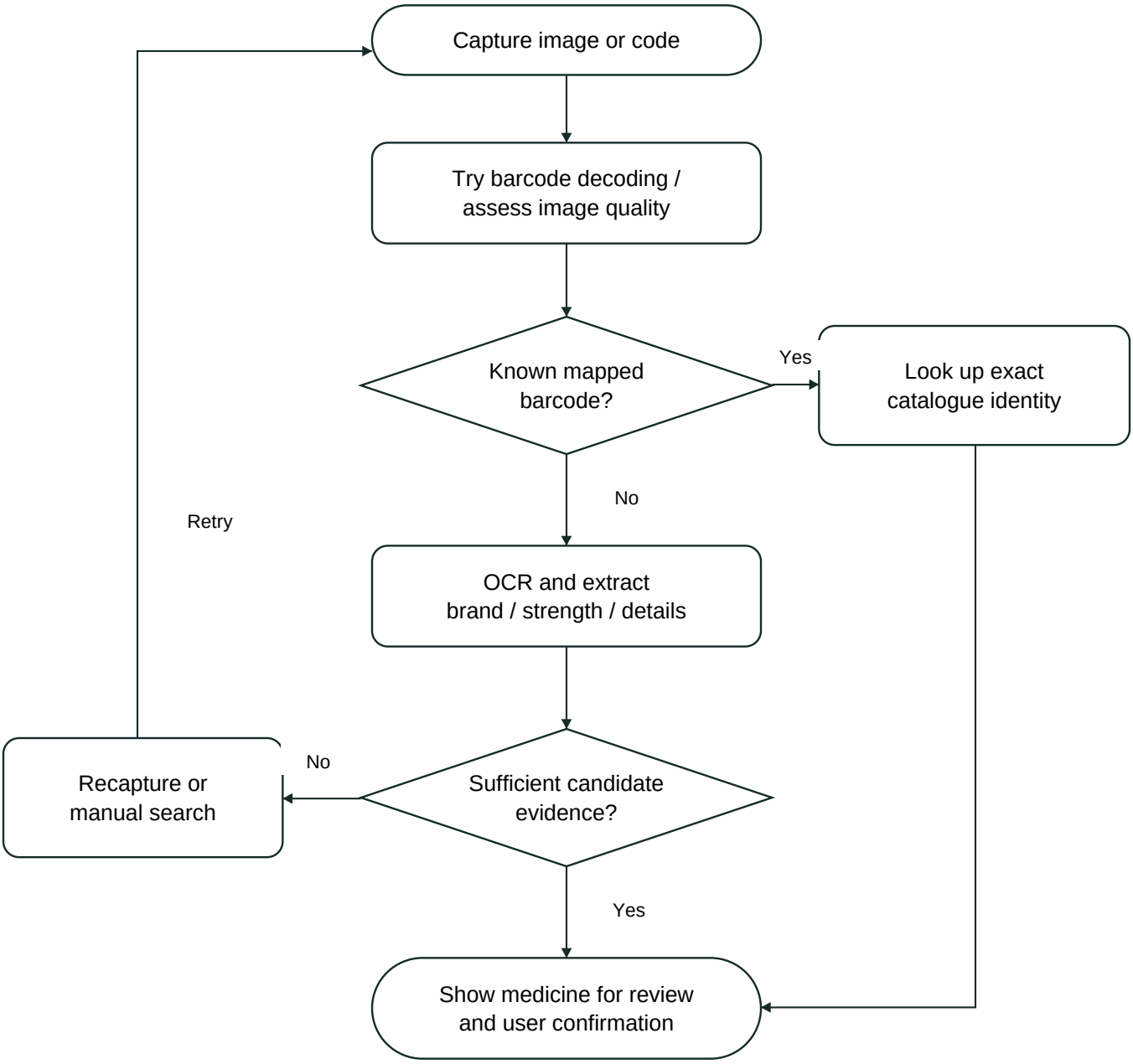


Figure 3.7 Barcode and photo recognition decision flow

Photo recognition is a working review-assisted input path. The image-quality result indicates blur/glare assessment, not guaranteed readable text. Low evidence produces a recovery path. A candidate score is a matching score, not a calibrated probability of medical correctness.

Functional Flow

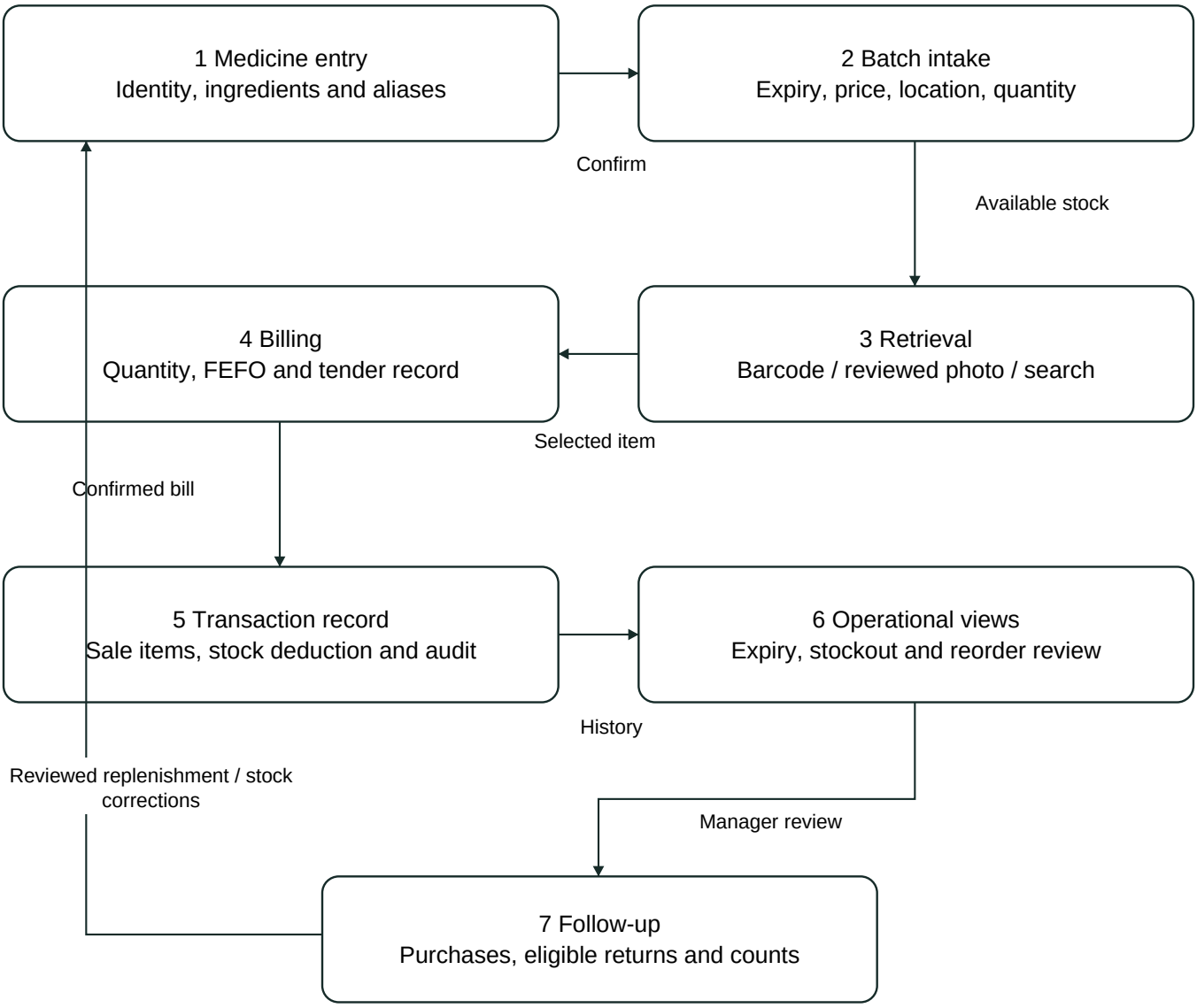


Figure 3.8 Functional flow from stocking to operational review

The same medicine identity links stocking and billing. Product photographs support entry and retrieval, while authoritative quantity comes from accepted batch/location records. A reorder suggestion creates a draft only when a user requests it; it does not place a supplier order.

Core Data Model

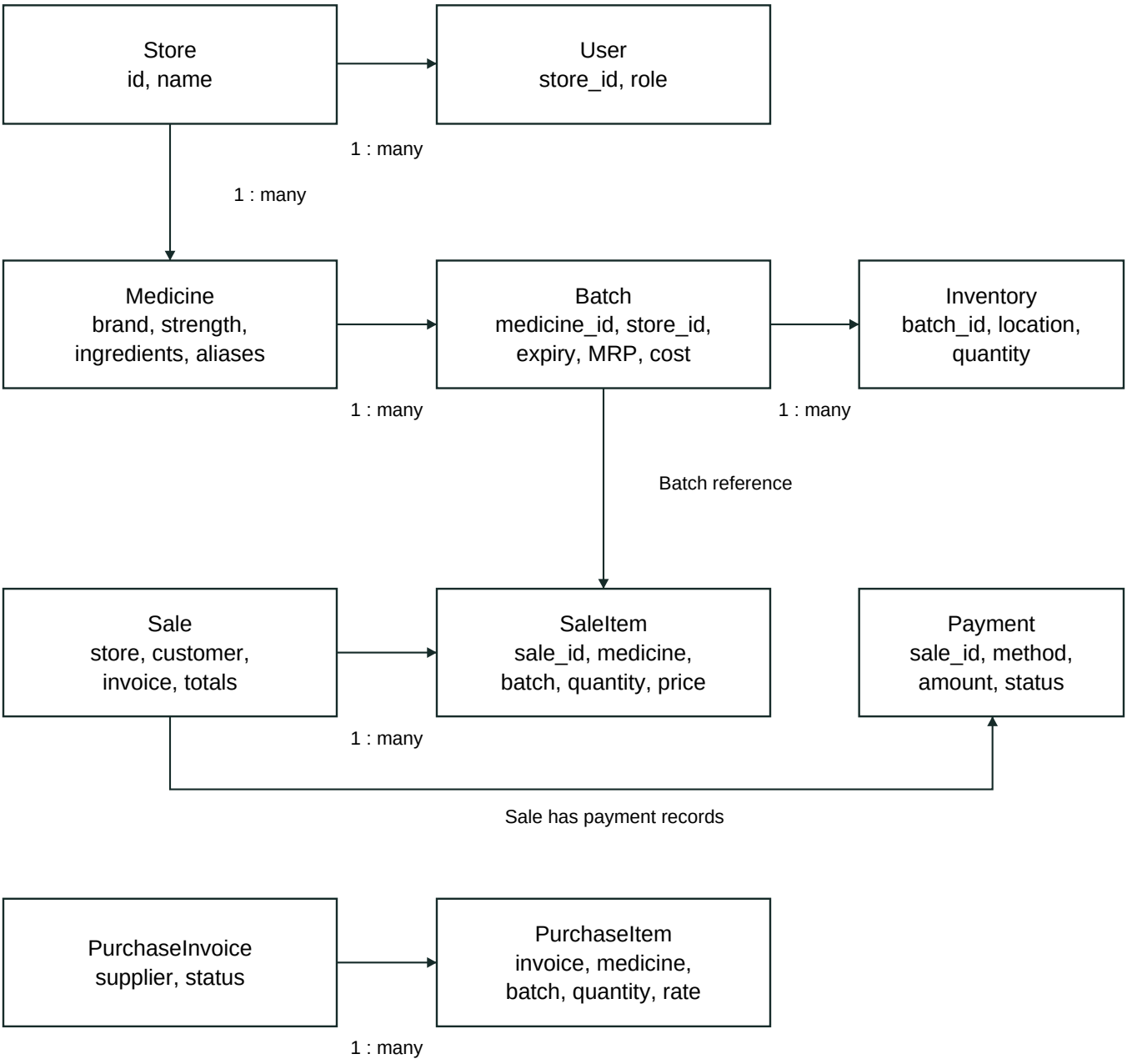


Figure 3.9 Core entity relationship model

The diagram is a logical subset, not an exhaustive schema. Medicine aliases support barcode identity. Batch numbers are unique within medicine and store. Inventory can be split across locations. Returns, audit logs, prescriptions, expenses and transfers extend these core records.

Algorithms and Transaction Rules

3.5 Identification and FEFO

Barcode lookup resolves a stored alias to a medicine. Packaging OCR extracts text and fields, then compares brand, strength and other evidence against catalogue candidates. The matcher exposes a score breakdown and uses a low-evidence recovery path. User verification is required before a recognised medicine is used in the bill.

For a valid sale, the backend filters stock by store and medicine, rejects expired or unavailable quantities and allocates eligible batches by expiry order. Store and inventory locks support a consistent write order. Currency values use decimal arithmetic. Sale, item, payment and stock changes are committed as one operation or rolled back on error.

3.6 Forecasting and Reorder Calculation

Demand forecasting uses recent daily sales. Sparse history uses an adaptive moving-average baseline; Ridge autoregression is selected only when the configured history condition is met, including at least fourteen nonzero selling days. Ridge features include previous-day demand, previous-week demand, a rolling weekly mean and weekday. Fit error measures are not independent test-set accuracy.

Target stock = ceiling[daily demand x (lead time + review period) + safety stock].
Net deficit = target stock - current eligible stock - open purchase quantity.
Suggested quantity = positive deficit rounded up to the configured pack size.

3.7 Reference Updates and Learning

Confirmed medicine/reference mappings can improve later retrieval. This feedback is distinct from retraining neural model weights. The project includes experimental photo-training utilities, but the deployed recognition workflow is not described as continuously retraining itself. A retrained model requires labelled data, independent evaluation and an explicit deployment decision.

Application APIs

The following endpoints are present in the inspected Flask route modules. Protected business routes require an authorised session and enforce store/role restrictions in the application. GET operations retrieve data; POST/PATCH/DELETE operations can alter state and require validation.

Area	Methods and route examples	Purpose
Authentication	POST /api/auth/login; GET /api/auth/me; POST /api/auth/users	Sign in, retrieve account and manage staff.
Medicine catalogue	GET/POST /api/medicines; GET /api/medicines/search	Search and maintain store medicine identities.
Barcode	GET /api/medicines/barcode; POST /api/medicines/barcode-image	Resolve a mapped barcode or decode an image.
Inventory	GET/POST /api/batches; POST /api/inventory/adjust	Record batches and audited quantity adjustments.
Expiry / location	GET /api/inventory/expiry; GET /api/locations; GET /api/shelf/layout	Review expiry and physical storage.
Sales	GET/POST /api/sales; GET /api/sales/{id}	Create bills and retrieve transaction details.
Returns	GET /api/returns/eligible-items/{reference}; POST /api/returns	Check return eligibility and record reviewed returns.
Purchases	GET/POST /api/purchases; POST /api/purchases/{id}/accept	Maintain drafts and accept verified intake.
Suppliers / expenses	GET/POST /api/suppliers; GET/POST /api/expenses	Maintain supplier and operating-expense records.
Branches / audit	GET /api/multibranch/stores; GET /api/audit-logs	Inspect branches and operation history.

Table 3.1 Business API inventory

Recognition APIs and Integrations

API group	Route examples	Function
Vision	POST /api/vision/recognize; POST /api/vision/verify	Produce and verify packaging candidates.
Stocking	POST /api/vision/stocking/read, /search, /confirm	Read, search and confirm reviewed entry.
Distributor invoice	POST /api/distributor/scan-invoice; /verify-delivery; /confirm-delivery	Extract invoice rows and verify delivery.
Prescription	POST /api/ai/prescription/scan; POST /api/prescriptions/{id}/verify	Draft transcription and pharmacist review.
Planning / copilot	GET /api/ml/reorder-recommendations; POST /api/ai/copilot/query	Read estimates and query grounded records.
Offline sync	GET /api/sync/offline-pack; POST /api/sync/replay	Account-scoped catalogue and idempotent replay.

Table 3.2 Recognition and support APIs

3.8 External and Platform Integrations

Google Gemini: server-side Google Gen AI SDK for optional handwriting transcription and copilot synthesis. The selected prescription image can contain patient information; consent and service configuration are required. Output is validated and reviewed.
Tavily: optional medicine-reference web search through its search API. Web results are reference candidates, not automatic stock or medical truth.
Aiven: MySQL business persistence and PostgreSQL AI reference/vector persistence, accessed by the backend.
Google Cloud: Cloud Run services, Cloud Build, Artifact Registry and Secret Manager for deployment and configuration.

Browser integrations: ZXing camera barcode decoding, media capture, optional speech recognition and PWA installation/service-worker APIs. These depend on browser capabilities and permission. Cash, UPI and card selections record tender information; no automatic bank settlement or payment-gateway integration is claimed.

CHAPTER 4

Project Planning

4.1 Work Breakdown and Suggested Milestones

The work is organised into requirements and schema, transactional services, catalogue and stock screens, assisted input, billing and returns, planning views, deployment and verification. The schedule below is a proposed academic organisation of work; it is not a retrospectively measured work log.

Work package	W1-2	W3-4	W5-6	W7-8	W9-10	W11-12
Requirements and data design	X	X				
Core stock and APIs		X	X			
Billing and procurement UI			X	X		
OCR and review flows				X	X	
Deployment and verification					X	X
Report and demonstration						X

Table 4.1 Suggested semester milestone chart

4.2 Team Participation

The project group consists of Neev Borana, Romit Meher and Rohit Gorhekar. Individual task ownership should be confirmed by the group and project guide before the viva. This report does not assign unverified development contributions to a student.

4.3 Risks and Mitigation

Risk	Impact	Mitigation
Ambiguous label or handwriting	Wrong candidate	Reject weak matches; retain original image and manual review.
Interrupted network / repeated request	Duplicate or pending bill	Persistent UUID queue and server replay receipt.
Thin sales history	Unstable estimate	Baseline fallback and user-reviewed draft orders.
Credential exposure	Unauthorised access	Keep secrets outside code; rotate exposed credentials.

Table 4.2 Project risk assessment

CHAPTER 5

Resources and Feasibility

Component	Inspected implementation or requirement
Frontend	Next.js 16.3.5, React 19.2.8, TypeScript 5 and responsive CSS.
Backend	Python Flask, SQLAlchemy, Alembic migrations and a production WSGI server.
Vision / ML	PaddleOCR, OpenCV, NumPy, scikit-learn; optional experimental PyTorch training.
Databases	Aiven MySQL for business data; Aiven PostgreSQL for AI references/vector data.
Hosting	Separate frontend/API containers on Google Cloud Run over HTTPS.
Capture / devices	Desktop or phone browser with camera permission; optional USB keyboard-style barcode scanner.
Development GPU	RTX 3070 available for local experiments; not a requirement for Cloud Run CPU inference.

Table 5.1 Software and hardware resources

5.1 Technical and Operational Feasibility

The prototype reuses established web, database and OCR libraries. Manual search and review permit the core workflow to continue when optional recognition or search cannot complete. Staff still need training in batch entry, stock units, return conditions and reconciliation. Real-world concurrency, data retention and recovery procedures need operational testing.

5.2 Economic Feasibility and Budget Basis

No verified procurement invoices or cloud billing statements are included. Existing computers and a camera can be reused. Costs depend on Cloud Run CPU/memory use, Aiven plans, image volume, model quotas, backups and printing/binding. Free API tiers may be useful for a classroom demonstration but are not a permanent cost guarantee. Before sustained use, set budget alerts and record the actual monthly service charges. No fabricated project expenditure is reported.

CHAPTER 6

Verification Methodology

Verification separates deterministic business tests, browser interaction checks and recognition observation. Stock and payment tests run against isolated fixtures. Deployed navigation checks are read-only and do not submit real payments, dispatch purchase orders or alter live stock.

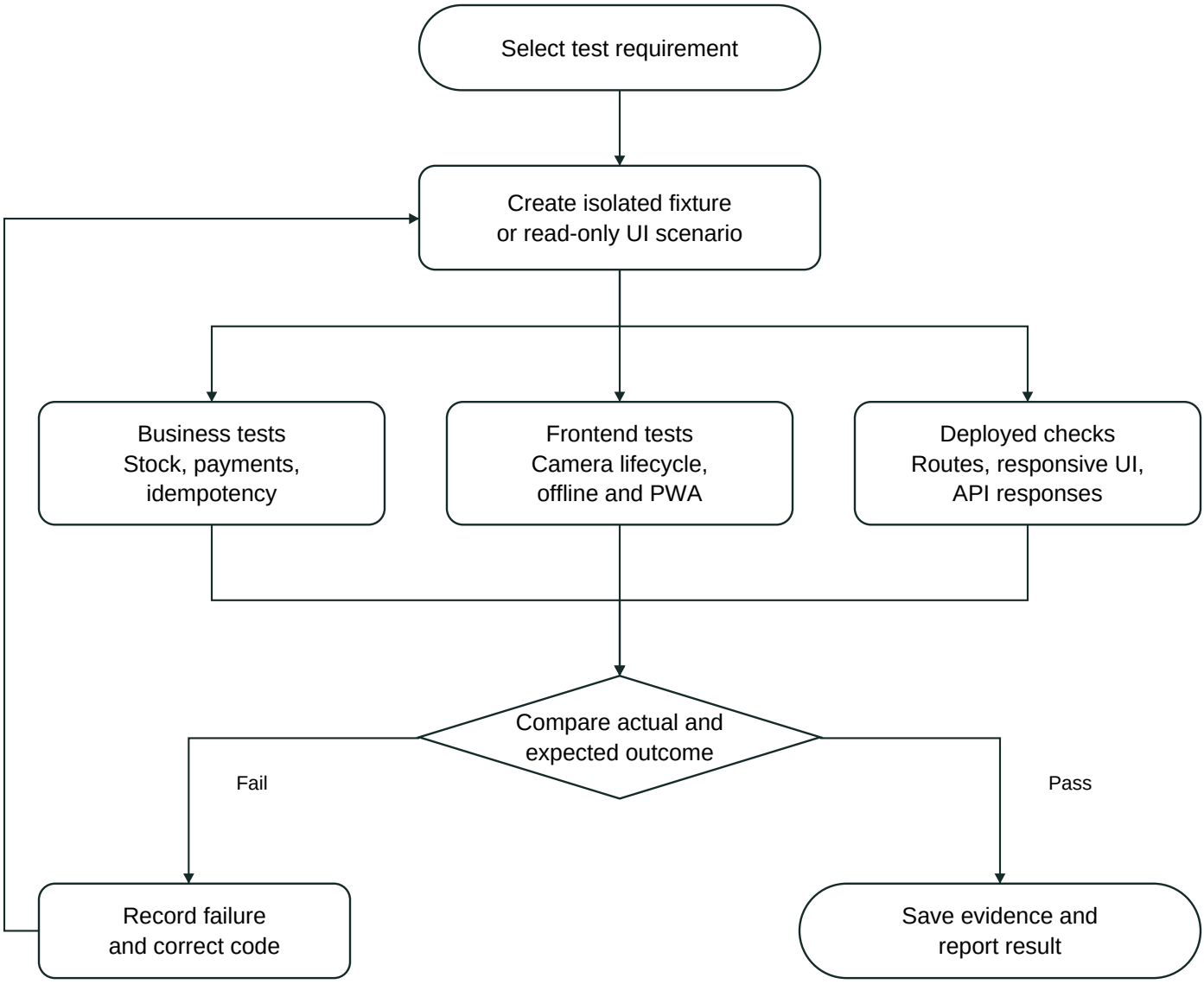


Figure 6.1 Test-case verification and correction flow

The pass branch records only outcomes actually observed. A code-level success does not substitute for an independent medicine-image accuracy study, physical-device scan test or production load test.

Test Cases and Expected Outcomes

ID	Input or action	Expected result	Evidence status
TC01	Import the eight-medicine demo fixture twice.	Second run creates no duplicate sales or stock.	Automated regression passed.
TC02	Compute imported stock and payment totals.	Stock is 480 - 217 + 20 = 283; line/payment totals balance.	Automated fixture passed.
TC03	Forecast a short series of selling days.	Use baseline; avoid recursive thousands-of-units suggestion.	Automated regression passed.
TC04	Open dashboard with stored demo activity.	Render chart, stock/expiry alerts and purchase drafts.	Live browser observed.
TC05	Open return lookup at 375 and 1440 px.	Icon does not overlap text; no page-level overflow.	Live dimensions checked.
TC06	Present mapped barcode frames; repeat same code.	Resolve code, suppress duplicate add and stop camera stream on close.	Simulated browser/component checks; physical device retest needed.
TC07	Scan a weak or rotated package photo.	Show review candidate or explicit recapture/manual fallback.	Workflow observed; general accuracy not quantified.
TC08	Load protected deep link offline.	Public reconnect screen; do not cache private response.	Prior PWA browser evidence.
TC09	Transcribe handwritten prescription.	Editable rows, warnings for ambiguity, no invented quantity.	Sample workflow observed; pharmacist review required.

Table 6.1 Representative test cases

TC02 describes the isolated seed fixture and imported demo batches, not a guarantee that all live inventory remains unchanged after later sales. Test cases describe their own scope. The screenshot set documents visible frontend screens, not successful execution of every backend operation.

Observed Results and Demonstration Data

6.1 Verification Evidence

The dashboard import and forecasting regression group completed with **7 tests passed**. The frontend test suite completed with **28 tests passed**. The production frontend build, including TypeScript checking, passed. ESLint reported **0 errors and 81 existing warnings**. These are scoped development checks recorded during this iteration, not a claim that every project path has been exhaustively tested.

Read-only deployed checks confirmed eight active catalogue medicines, populated dashboard insights and a seven-day recent-invoice chart. Return lookup measurements at 375 px and 1440 px showed a 14 px gap between the icon edge and the start of input text, with no page-level overflow in that check. A candidate deployment returned finite reorder quantities after the sparse-history correction.

6.2 Demonstration Dataset

Dataset item	Prepared demonstration content
Medicines and photographs	8 catalogue medicines with 16 paired front/back photographs.
Sales and people	50 added synthetic sales; 12 fictional customers and 3 fictional suppliers.
Procurement and expenses	5 purchase invoices, including 3 drafts; 6 fictional operating expenses.
Stock movement	217 original demo units sold through recorded sales; 20 units newly received in 3 near-expiry batches.
Purpose	Populate operational examples without presenting them as real pharmacy activity.

Table 6.2 Prepared classroom demonstration data

6.3 Interpretation

Photo recognition functions as an assisted workflow that returns evidence for review or a recoverable no-match state. The current evaluation does not establish a clinical or general packaging identification accuracy percentage. Performance and recognition depend on image orientation, lighting, readable text and available reference data.

CHAPTER 6 FRONTEND IMPLEMENTATION

Sign In

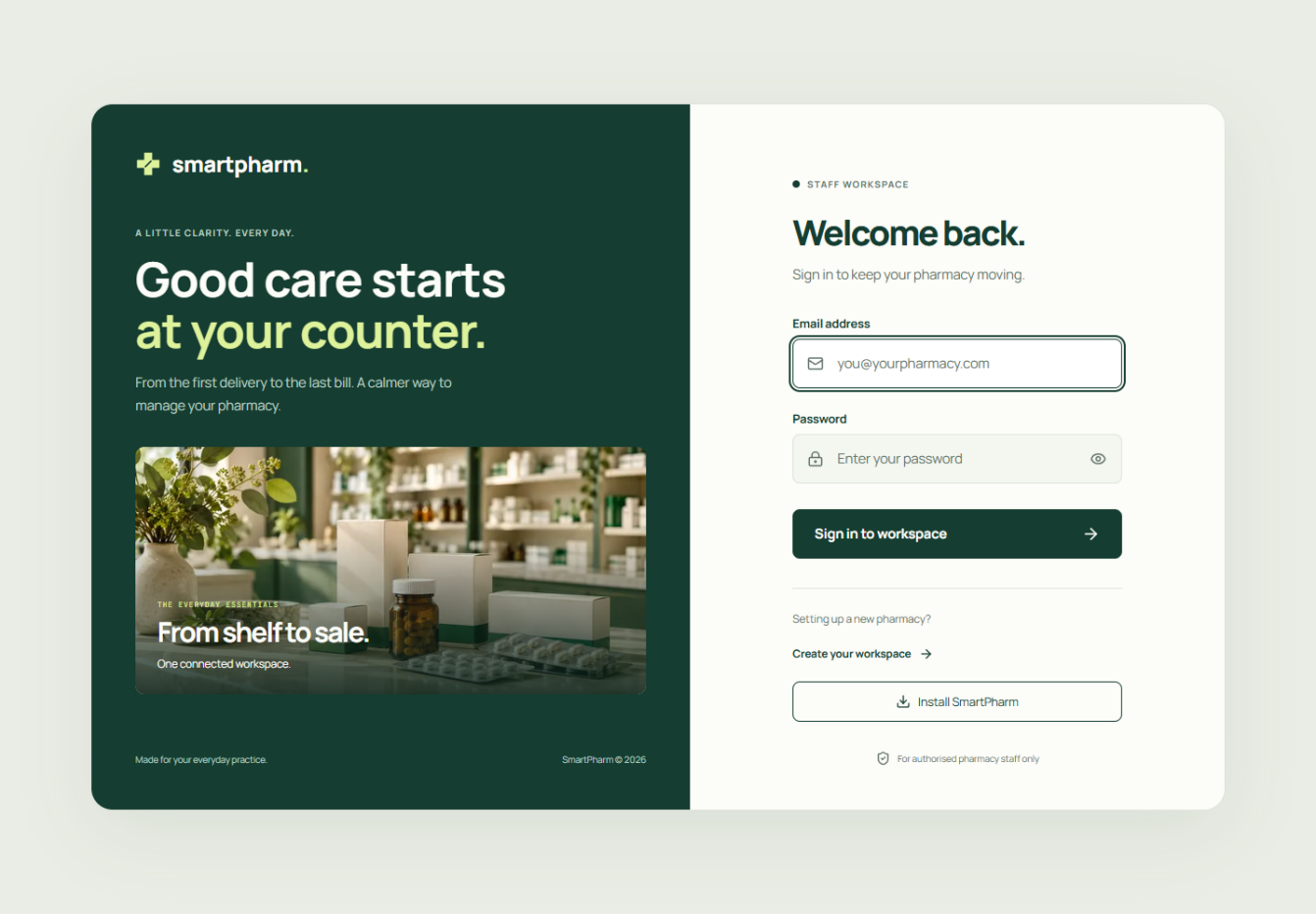


Figure 6.2 Sign In actual deployed frontend

The sign-in page provides staff authentication, a password visibility control and workspace setup access. The image shows the actual deployed page with no credentials entered.

Users authenticate before protected records are retrieved. A screenshot of a login form does not establish penetration-test coverage.

Capture context: deployed HTTPS application, desktop browser viewport 1440 x 1000, 26 October 2026. Fictional classroom records are used where available.

CHAPTER 6 FRONTEND IMPLEMENTATION

Dashboard

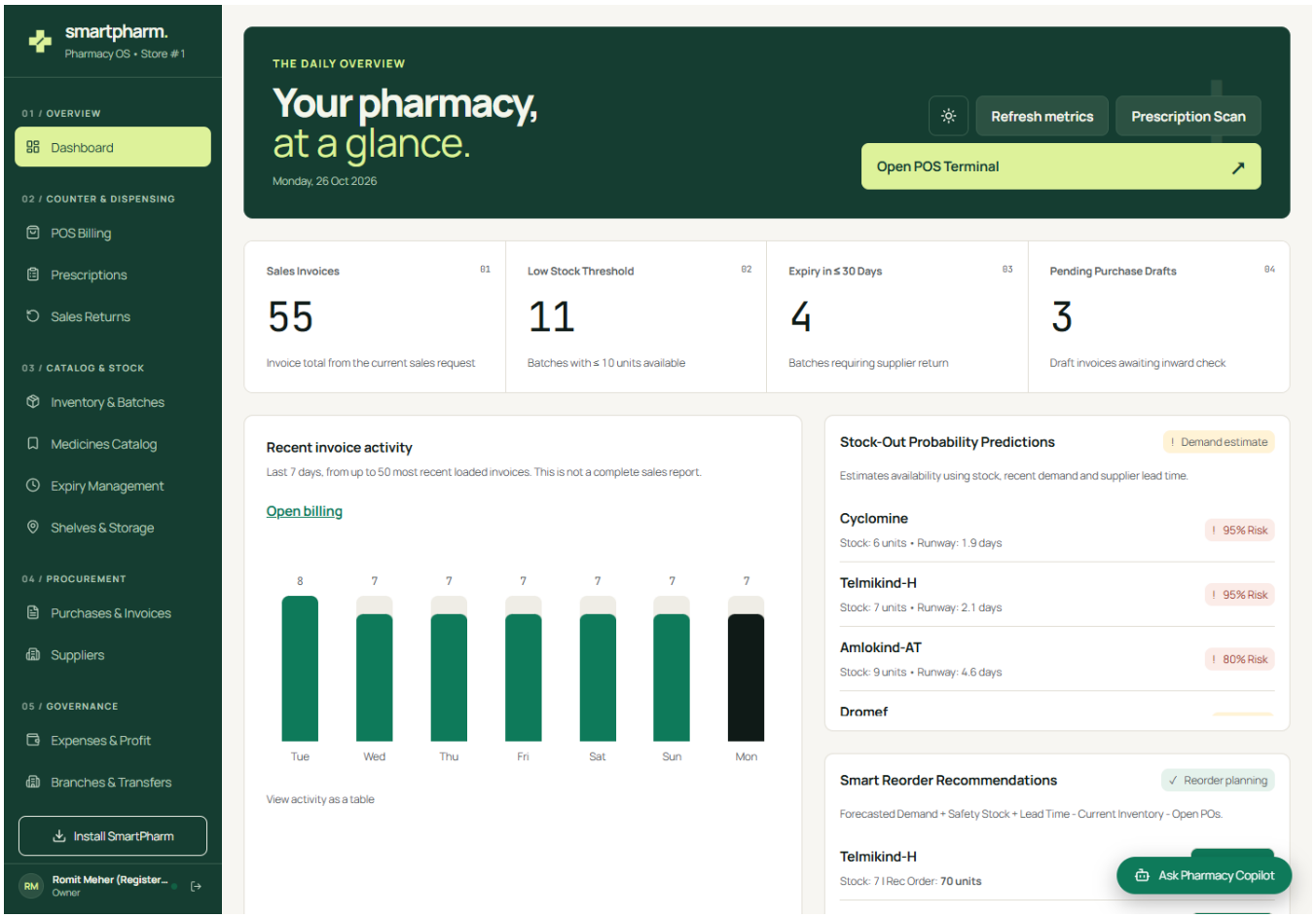


Figure 6.3 Dashboard actual deployed frontend

The dashboard combines recent invoice activity, low-stock and expiry indicators, pending purchase drafts and inventory planning insights. The screenshot uses stored demonstration activity.

The chart covers up to fifty recently loaded invoices rather than a complete sales report. Forecast and reorder values are estimates; they do not place orders automatically.

Capture context: deployed HTTPS application, desktop browser viewport 1440 x 1000, 26 October 2026. Fictional classroom records are used where available.

CHAPTER 6 FRONTEND IMPLEMENTATION

POS Billing Terminal

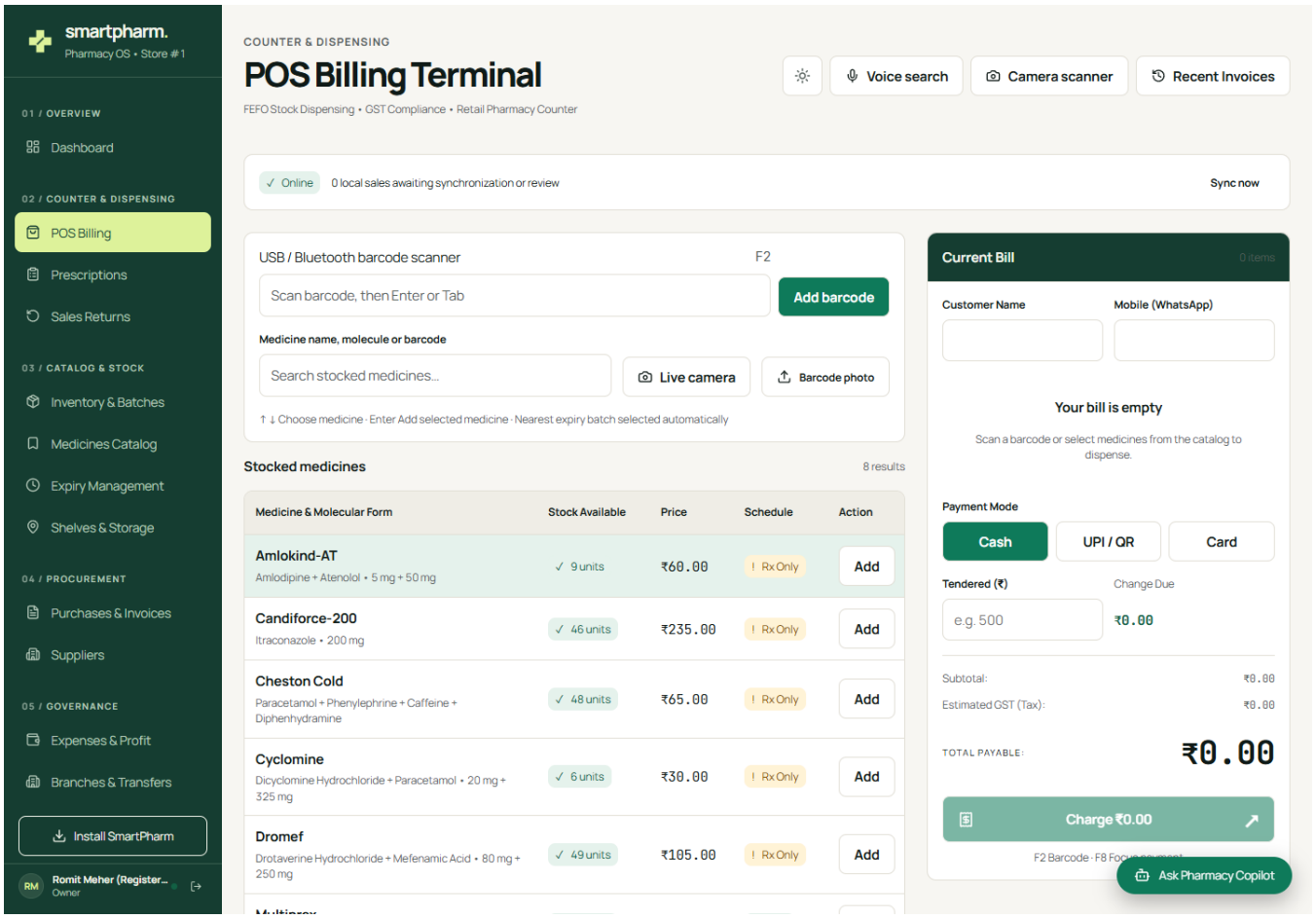


Figure 6.4 POS Billing Terminal actual deployed frontend

The counter screen provides barcode entry, catalogue search, camera/photo actions and the current bill. Cash, UPI and card choices describe recorded tender methods.

FEFO allocation and stock checks occur at the backend. Barcode or photo selection must identify the intended medicine before checkout. This capture does not submit a payment.

Capture context: deployed HTTPS application, desktop browser viewport 1440 x 1000, 26 October 2026. Fictional classroom records are used where available.

CHAPTER 6 FRONTEND IMPLEMENTATION

Inventory and Batches

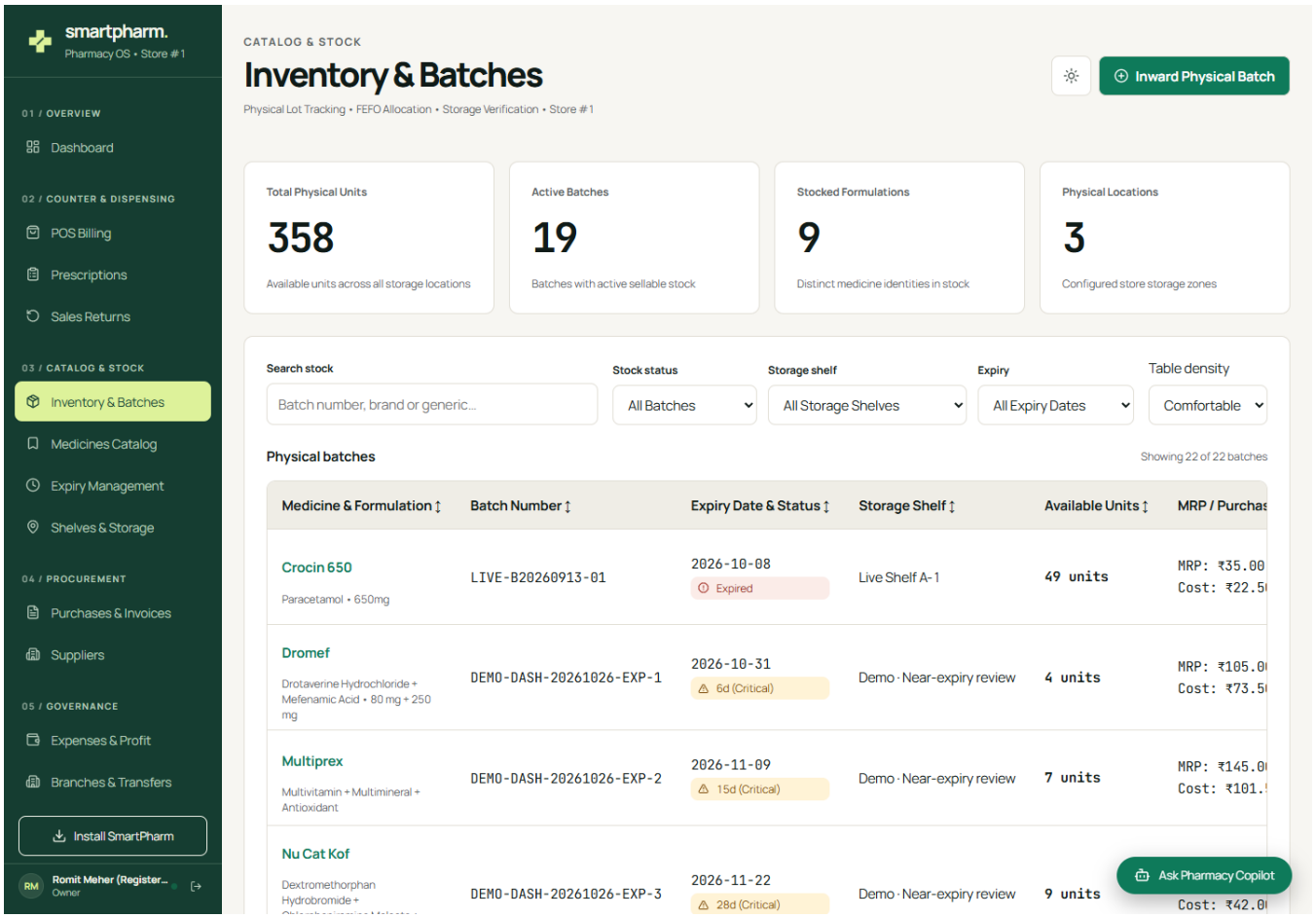


Figure 6.5 Inventory and Batches actual deployed frontend

The inventory workspace exposes batch-specific stock details and supports review of quantity, expiry and location. Medicine identity is kept separate from the stock record.

A batch may have its own MRP and purchase cost. Staff should reconcile physical counts before confirming an adjustment; the report capture performs no stock write.

Capture context: deployed HTTPS application, desktop browser viewport 1440 x 1000, 26 October 2026. Fictional classroom records are used where available.

CHAPTER 6 FRONTEND IMPLEMENTATION

Medicine Catalogue

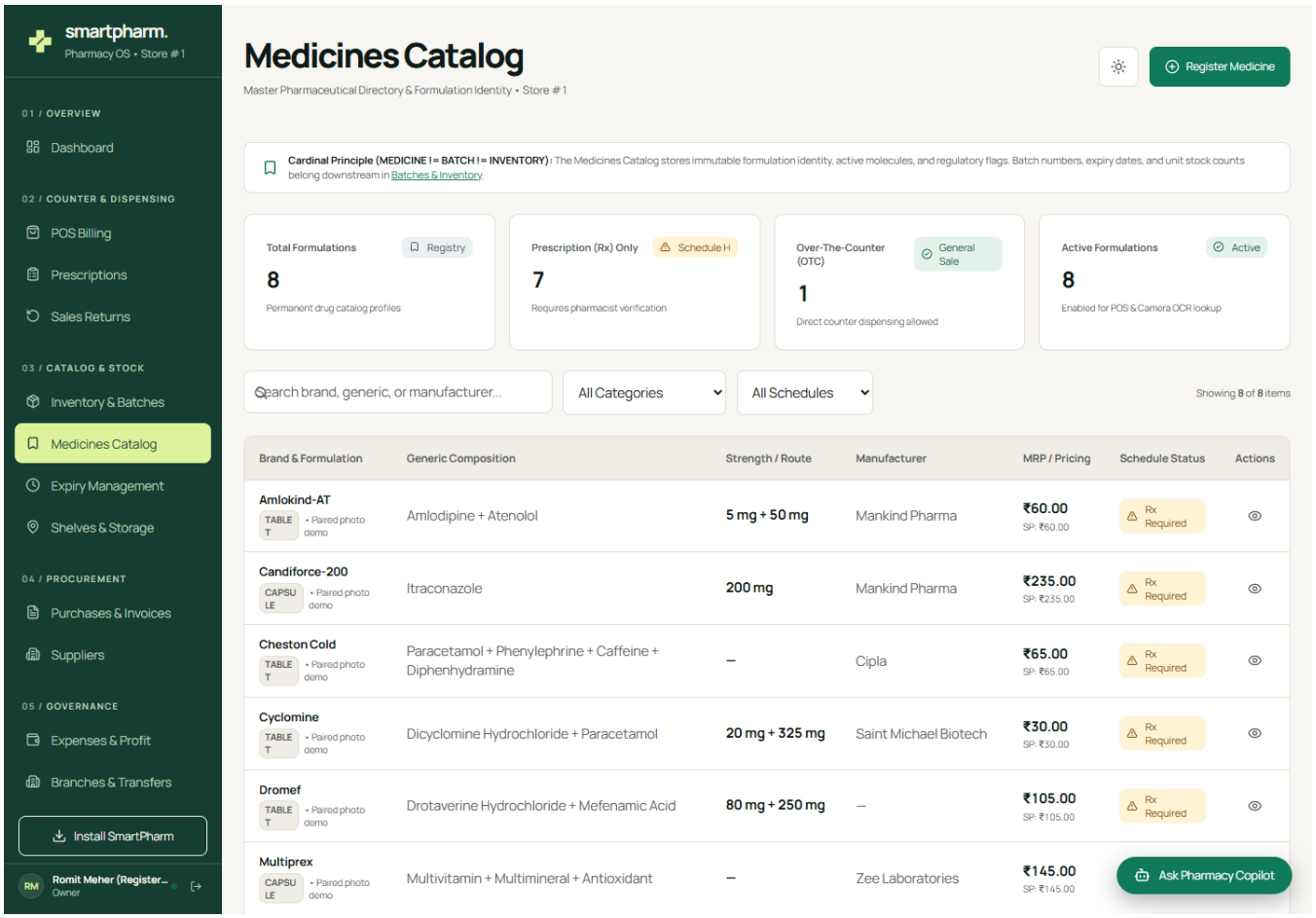


Figure 6.6 Medicine Catalogue actual deployed frontend

The catalogue screen presents the eight active demonstration medicines and their stored identity details. Search and reviewed entry connect this list to stocking and billing.

The eight brands are Amlokind-AT, Candiforce-200, Cheston Cold, Cyclomine, Dromef, Multiprex, Nu Cat Kof and Telmikind-H. Package/strength details must be verified during entry.

Capture context: deployed HTTPS application, desktop browser viewport 1440 x 1000, 26 October 2026. Fictional classroom records are used where available.

CHAPTER 6 FRONTEND IMPLEMENTATION

Purchases and Invoices

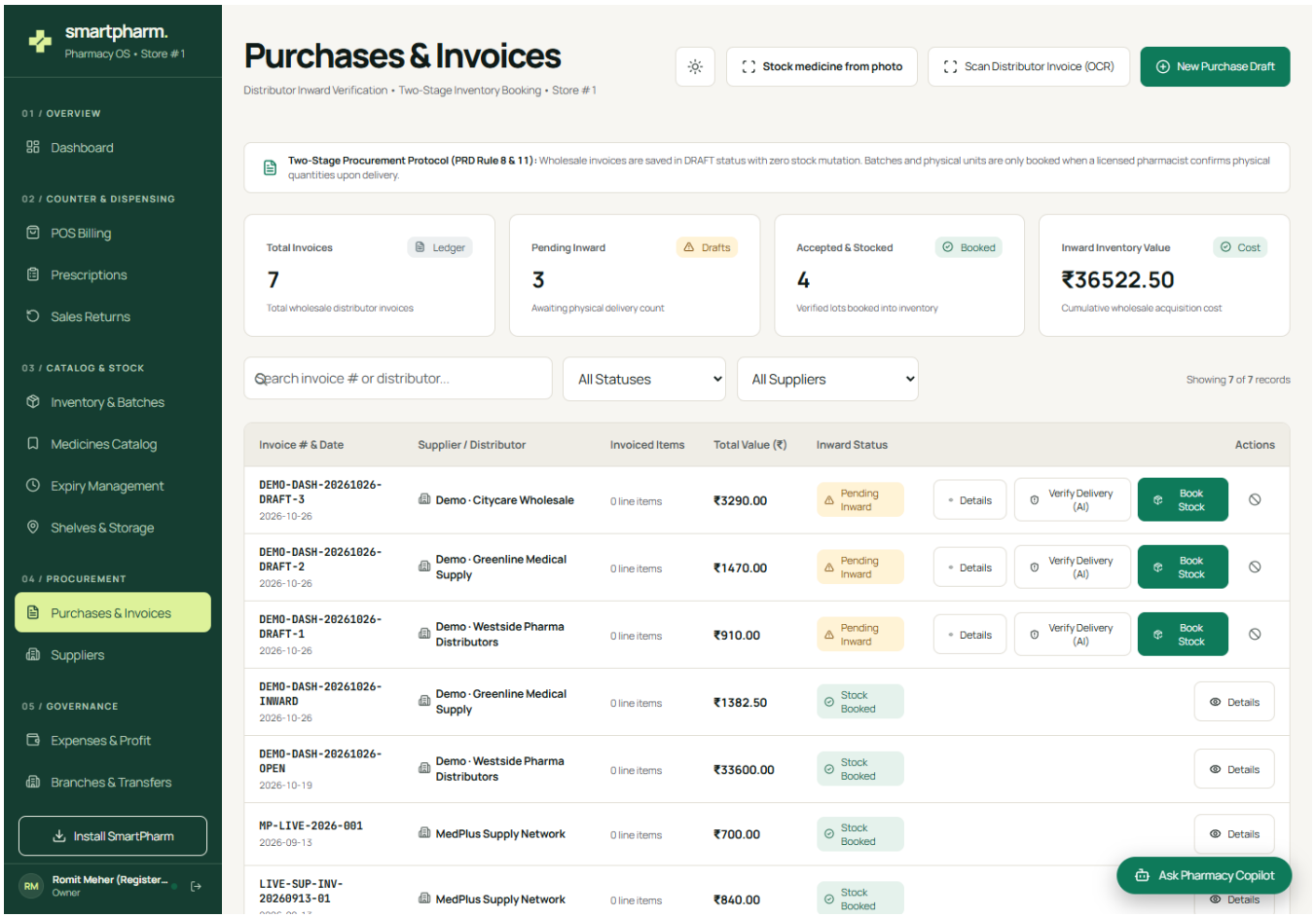


Figure 6.7 Purchases and Invoices actual deployed frontend

Purchase records distinguish draft review from accepted intake. Invoice digitisation supports candidate extraction, delivery comparison and confirmation.

The demo includes three pending drafts. A draft is not accepted stock, and a suggested purchase is not evidence of a supplier order or payment.

Capture context: deployed HTTPS application, desktop browser viewport 1440 x 1000, 26 October 2026. Fictional classroom records are used where available.

CHAPTER 6 FRONTEND IMPLEMENTATION

Prescription Workspace

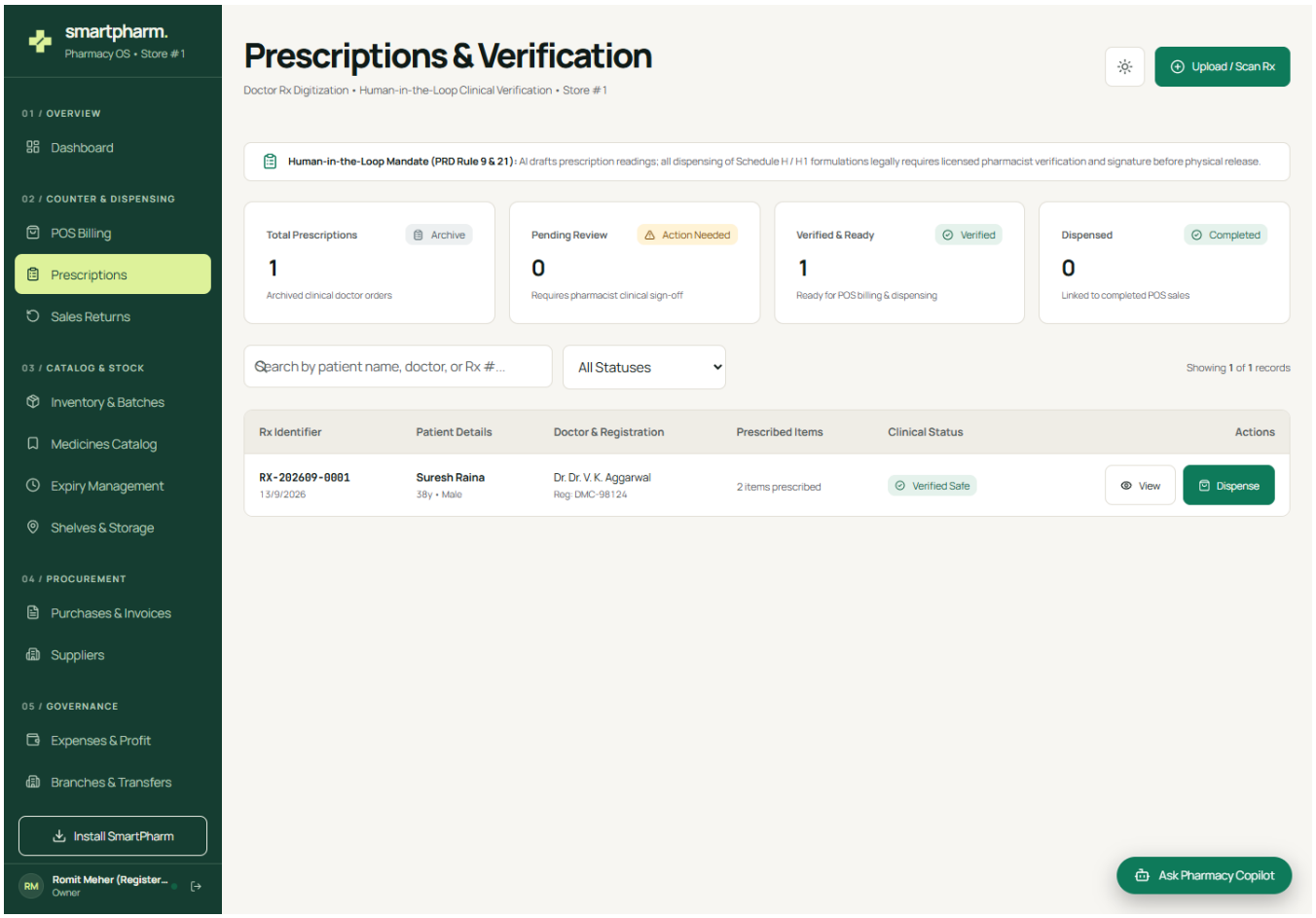


Figure 6.8 Prescription Workspace actual deployed frontend

Prescription capture and digitisation support text or image input with review before verification and dispensing. Handwriting assistance produces editable draft rows.

Google handwriting mode sends the selected image to a third-party model service. Ambiguous fields need human correction. No real patient image is embedded in this report.

Capture context: deployed HTTPS application, desktop browser viewport 1440 x 1000, 26 October 2026. Fictional classroom records are used where available.

CHAPTER 6 FRONTEND IMPLEMENTATION

Expiry Management

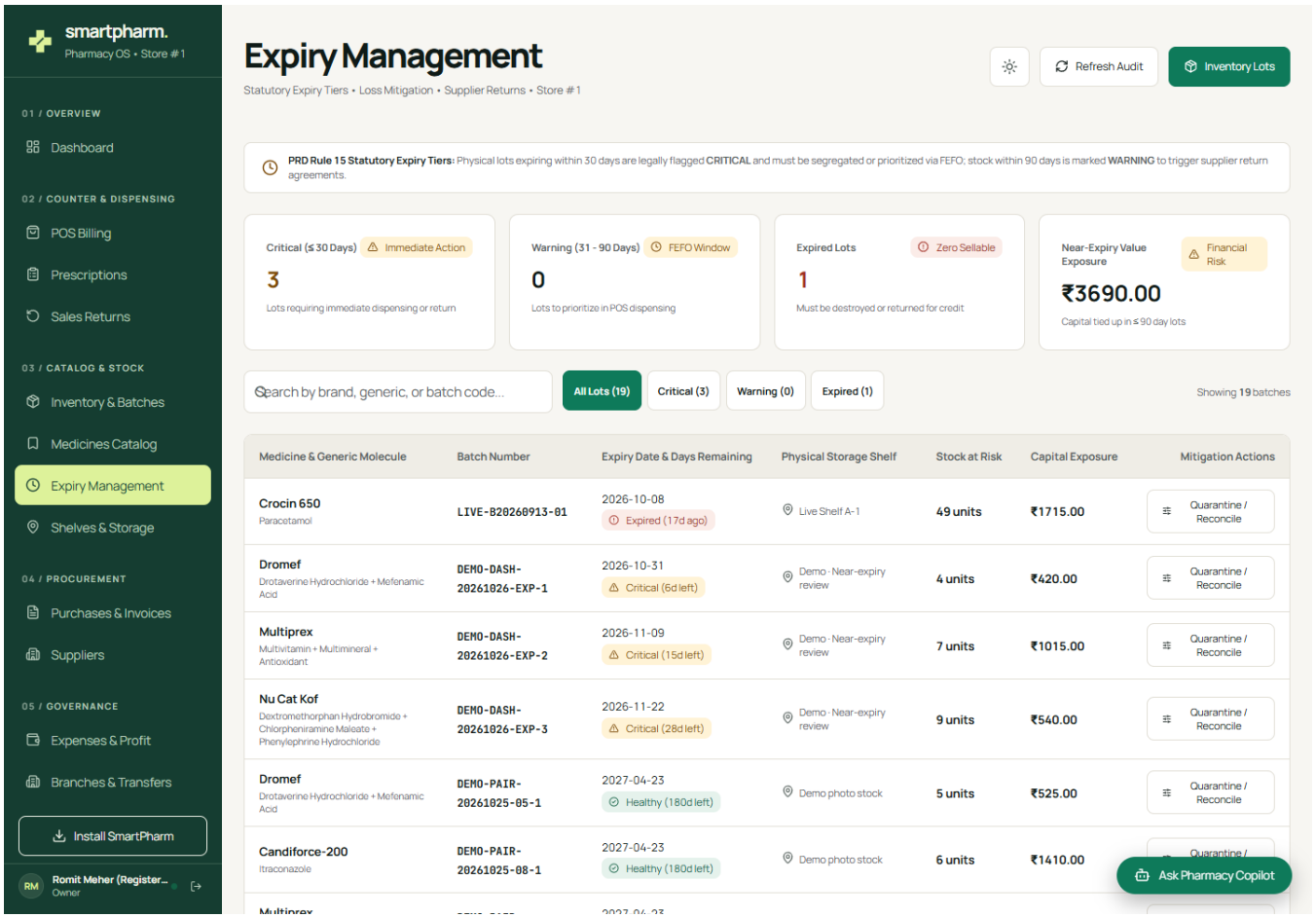


Figure 6.9 Expiry Management actual deployed frontend

The expiry workspace highlights batches that need review according to their recorded expiry and remaining stock. The demonstration includes deliberately prepared near-expiry examples.

Recorded expiry controls stock eligibility. Alerts support staff review and supplier-return planning; a dashboard count is not an independent physical inspection.

Capture context: deployed HTTPS application, desktop browser viewport 1440 x 1000, 26 October 2026. Fictional classroom records are used where available.

CHAPTER 7

Sustainability and Impact

7.1 Industrial Relevance

Batch-level record keeping, expiry review and consistent sales history are useful operational capabilities for a physical pharmacy. The project demonstrates how procurement and billing can share a medicine identity while preserving different batch prices and expiry dates. Audit history supports investigation of stock changes, subject to proper staff practices and reliable records.

7.2 Societal Impact

Clear identification and review screens can help staff detect data-entry mistakes before a transaction. Prescription and packaging assistance reduce repeated typing but do not replace professional judgement. Accessibility, simple recovery paths and mobile layouts are relevant to staff working at busy counters. Benefits are potential operational outcomes, not measured improvements in patient health.

7.3 Environmental Impact

Expiry visibility can support timely review and better replenishment planning, potentially reducing avoidable disposal. Digital receipts and records can reduce routine printing where accepted. Hosting and model inference also consume resources; smaller public caches, bounded image handling and reuse of existing equipment help limit unnecessary work. No lifecycle carbon assessment has been performed.

7.4 SDG Mapping

Goal	Connection to project	Evidence boundary
SDG 3 Good Health and Well-being	Organised medicine records and pharmacist verification.	Supports a workflow; no clinical outcome claim.
SDG 9 Industry, Innovation and Infrastructure	Connected web and data infrastructure for operations.	Academic engineering prototype.
SDG 12 Responsible Consumption and Production	Expiry review and demand-informed stock planning.	Waste reduction has not been measured.

Table 7.1 Qualitative sustainability mapping

CHAPTER 8

Conclusion and Future Scope

8.1 Conclusion

SmartPharm connects catalogue management, batch-level stock, procurement, POS billing, returns and operational insight in a deployed web application. Its core design keeps medicine identity separate from batch and location quantities. Reviewed barcode/photo input and document digitisation connect to these business records rather than acting as independent sources of stock truth.

The project demonstrates functioning review-assisted recognition, FEFO stock allocation, editable document extraction, account-scoped offline transaction handling and a PWA installation/reconnect experience. Verification supports the specific behaviours and checks described in Chapter 6. Synthetic records make the demonstration repeatable without representing fictional activity as commercial sales.

8.2 Future Scope

Recognition work should add orientation-aware evaluation, consistent package cropping and a larger independently labelled image test set. A public benchmark should report accepted-match precision, recall, rejection rate and end-to-end latency. Confidence calibration should be assessed before interpreting a score as probability.

Operational work should include MySQL concurrent-terminal stress tests, tested backup restoration, stronger access review and a documented privacy/retention policy for uploaded prescriptions. Physical phones and USB scanners should be tested under the actual classroom lighting and network conditions. Browser support should be recorded by device and operating system.

Forecasting should be validated on held-out time periods and longer real demand histories before replenishment estimates are used operationally. Optional payment gateways, supplier ordering and regulatory integrations would require separate contracts, security review and transactional reconciliation. Neither a demo tender record nor a reference web result is equivalent to such an integration.

References

[1] B. Shi, X. Bai and C. Yao, "An End-to-End Trainable Neural Network for Image-based Sequence Recognition and Its Application to Scene Text Recognition," arXiv:1507.05717, 2015. https://arxiv.org/abs/1507.05717

[2] M. Liao, Z. Wan, C. Yao, K. Chen and X. Bai, "Real-time Scene Text Detection with Differentiable Binarization," AAAI, 2020; preprint arXiv:1911.08947, 2019. https://arxiv.org/abs/1911.08947

[3] Y. Du et al., "PP-OCR: A Practical Ultra Lightweight OCR System," arXiv:2009.09941, 2020. https://arxiv.org/abs/2009.09941

[4] M. Li et al., "TrOCR: Transformer-based Optical Character Recognition with Pre-trained Models," arXiv:2109.10282, 2021, revised 2022. https://arxiv.org/abs/2109.10282

[5] Y. Huang, T. Lv, L. Cui, Y. Lu and F. Wei, "LayoutLMv3: Pre-training for Document AI with Unified Text and Image Masking," ACM Multimedia, 2022, doi:10.1145/3503161.3548112. https://arxiv.org/abs/2204.08387

[6] C. Cui et al., "PaddleOCR 3.0 Technical Report," arXiv:2507.05595, 2025. https://arxiv.org/abs/2507.05595

[7] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," NeurIPS, 2020. https://arxiv.org/abs/2005.11401

Research metadata and original publication pages were checked on 26 October 2026. Survey findings are qualitative; paper benchmark scores have not been presented as SmartPharm results.

Implementation Sources

The implementation description is based on the inspected local repository and recorded project verification evidence as of 26 October 2026. These internal implementation sources distinguish actual project behaviour from the research literature.

Source	Relevant material
frontend/package.json; src/app and src/components	Framework versions, application routes, POS, staff interface, camera scanner and responsive layout.
backend/app/models; app/services	Medicine/batch/inventory relationships, FEFO sales, locks, decimal currency and transaction handling.
backend/app/api	Authentication, stock, vision, invoice, prescription, sales, returns, ML and sync routes.
backend/app/vision; app/ai	OCR pipeline, evidence matching, stocking references, prescription vision and grounded copilot.
backend/app/ml	Demand forecasts, stockout/expiry estimates and reorder calculation.
frontend/docs/pwa.md; pwa/sw-template.js	Installation controls, public-asset cache policy, reconnect fallback and update handling.
plans/dashboard-demo-20261026.md	Synthetic activity scope, stock balance, focused tests and deployment evidence.
output/dashboard-demo and output/capstone-report/assets	Browser layout measurements, dashboard evidence and eight captured frontend screens.

Table R.1 Implementation evidence inventory

External platform/library names identify integrations used or supported by the project. TrOCR, CRNN, DBNet and LayoutLMv3 are discussed in the literature survey; this report does not claim that every research architecture is directly deployed. Credentials, environment files and confidential connection details are intentionally excluded.

APPENDIX A

Examination and Demonstration Guide

A.1 Print and Submission

Prepare **two copies for the group**: one for the department and one for the students. Bring the report on the external examination day. Complete the blank guide, approval, date and signature fields according to college instructions. Confirm the prescribed binding, print size and final degree wording with the department before printing.

A.2 Suggested Demonstration Sequence

Step	Demonstration	Review point
1	Sign in and open the dashboard.	Explain that activity records are synthetic.
2	Open catalogue and batch inventory.	Show medicine identity versus batch/location stock.
3	Read a known barcode or search a stocked medicine.	Confirm product and strength before adding to bill.
4	Show packaging capture and the review screen.	Explain evidence, manual correction and the no-match path.
5	Open purchases and prescription workspace.	Show drafts and verification before accepting or dispensing.
6	Open expiry management and reorder view.	Explain expiry eligibility and estimates rather than automatic orders.
7	Show mobile layout / PWA installation action.	Installation and camera features depend on browser support.

Table A.1 Demonstration sequence

A.3 Preparation Checklist

Check the login and network in advance. Use the labelled demo package photos or known barcode cards. Confirm camera permission and keep manual search available. Do not display passwords, API keys, connection strings or a real patient prescription to the audience. Ensure pending local bills are reconciled before clearing browser data.